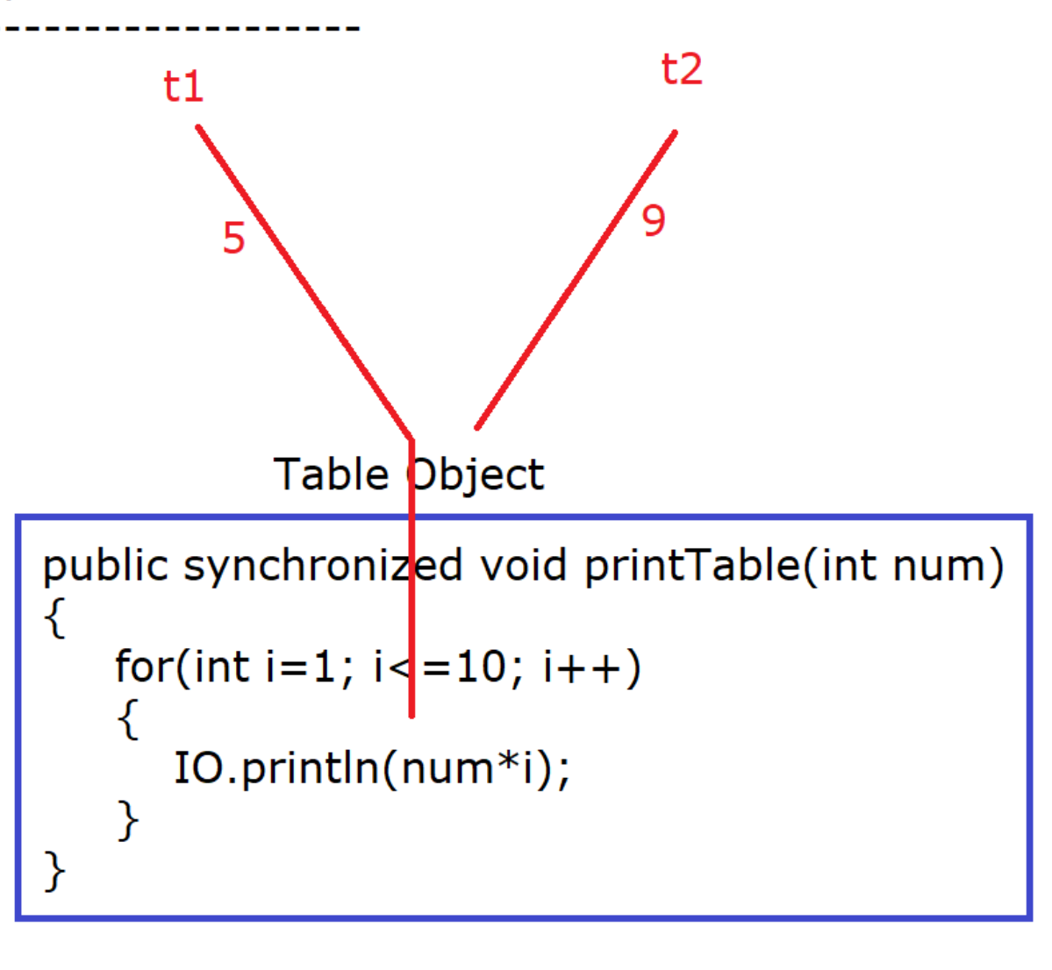
//Program on method level synchronization :



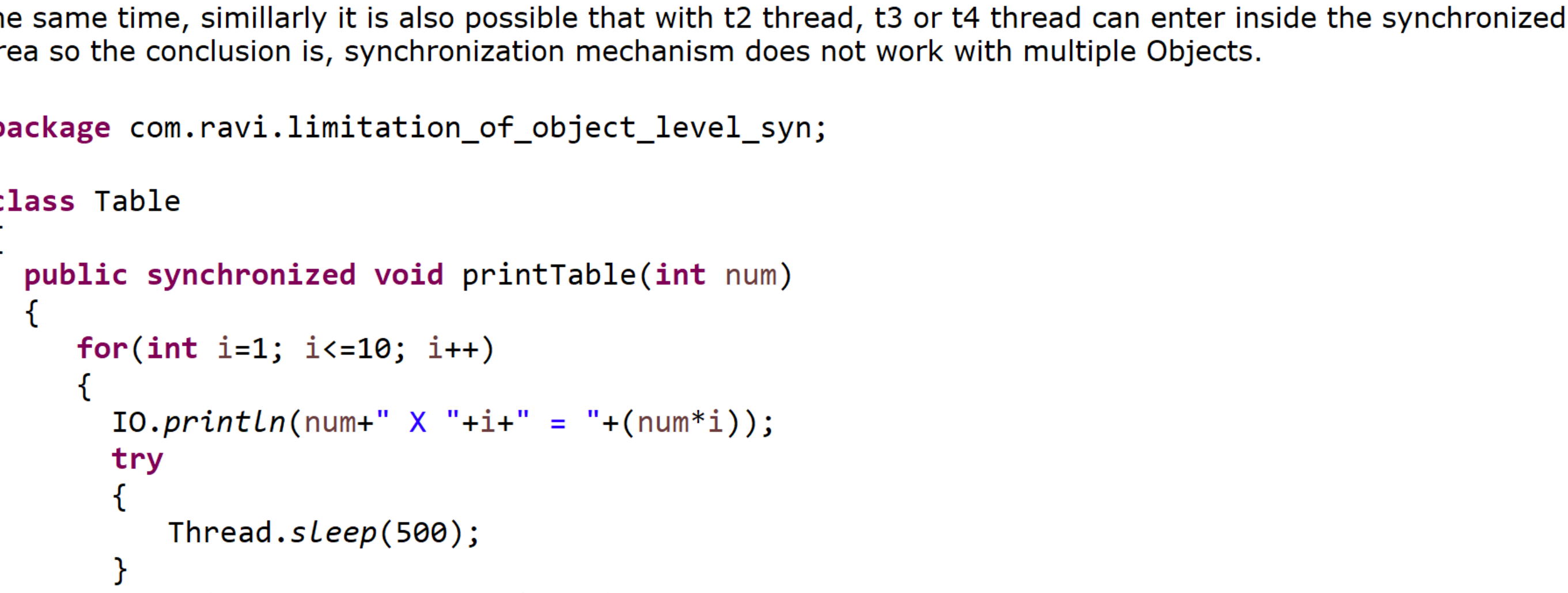
```
public synchronized void printTable(int num)
{
    for(int i=1; i<=10; i++)
    {
       IO.println(num*i);
    }
}
```

```java
package com.ravi.synchronization;

class Table
{
  public synchronized void printTable(int num)
  {
     for(int i=1; i<=10; i++)
     {
         IO.println(num+" X "+i+" = "+(num*i));

         try
         {
             Thread.sleep(1000);
         }
         catch(InterruptedException e)
         {
             e.printStackTrace();
         }

     }
     IO.println("........................");
  }
}

public class MethodLevelSynchronization {

    public static void main(String[] args)
    {
        Table obj = new Table();  //lock is created

        Thread t1 = new Thread()
        {
            @Override
            public void run()
            {
                obj.printTable(5);
            }
        };


        Thread t2 = new Thread()
        {
            @Override
            public void run()
            {
                obj.printTable(9);
            }
        };

        t1.start();  t2.start();

    }

}
```

//Program on block level Synchronization :

```java
package com.ravi.synchronization;

class ThreadName
{
    public void printThreadName()
    {
        //Free area for all the threads
        String name = Thread.currentThread().getName();
        IO.println("Running thread name is :"+name);

        synchronized(this)
        {
           for(int i=1; i<=10; i++)
           {
               IO.println(i+" by "+name+" thread");

               try
               {
                   Thread.sleep(1000);
               }
               catch(InterruptedException e)
               {

               }
           }
           IO.println(name +" thread has completed the task");
        }


    }
}


public class BlockLevel
{
    public static void main(String[] args)
    {
        ThreadName threadName = new ThreadName();  //lock is created

        Runnable r1 = () -> threadName.printThreadName();

        Thread t1 = new Thread(r1, "Child1");
        Thread t2 = new Thread(r1, "Child2");

        t1.start();  t2.start();
    }

}
```

Drawback OR Limitation of Object level synchronization :



```
public synchronized void printTable(int num)
 {
   for(int i=1; i<=10; i++)
   {
      IO.println(num*i);
   }
 }
```

```
public synchronized void printTable(int num)
 {
   for(int i=1; i<=10; i++)
   {
      IO.println(num*i);
   }
 }
```

From the above diagram it is clear that there is no interference between t1 and t2 thread because they are passing throgh Object1 where as on the other hand there is no interferenec even in between t3 and t4 threads because they are also passing through Object2 (another object).

But there may be chance that with t1 Thread (object1), t3 or t4 thread can enter inside the synchronized area at the same time, simillarly it is also possible that with t2 thread, t3 or t4 thread can enter inside the synchronized area so the conclusion is, synchronization mechanism does not work with multiple Objects.

```java
package com.ravi.limitation_of_object_level_syn;

class Table
{
  public synchronized void printTable(int num)
  {
     for(int i=1; i<=10; i++)
     {
       IO.println(num+" X "+i+" = "+(num*i));
       try
       {
          Thread.sleep(500);
       }
       catch(InterruptedException e)
       {
          e.printStackTrace();
       }
     }
     IO.println("===========================");
  }
}

public class DrawbackOfObjectLevelSyn {

    public static void main(String[] args)
    {
        Table obj1 = new Table(); //lock1
        Table obj2 = new Table(); //lock2


        Thread t1 = new Thread()
        {
           @Override
           public void run()
           {
               obj1.printTable(5);   //From Lock 1
           }
        };

        Thread t2 = new Thread()
        {
           @Override
           public void run()
           {
               obj1.printTable(9);   //From Lock 1
           }
        };


        Thread t3 = new Thread()
        {
           @Override
           public void run()
           {
               obj2.printTable(15);   //From Lock 2
           }
        };


        Thread t4 = new Thread()
        {
           @Override
           public void run()
           {
               obj2.printTable(19);   //From Lock 2
           }
        };


        t1.start();  t2.start();  t3.start();  t4.start();


    }

}
```

Note : From the above program it is clear that **synchronization logic** will not work with **multiple objects.**

In order to avoid the drawback of **Object level synchronization**, Java software people introduced static synchronization.

** Static Synchronization :

If we make our synchronized method as static then it is called static synchronization.

For static synchronization "Object is not required". Static method we can call with the help of class name.

In static synchronization, the thread will take the lock from the class but not from the Object. [Every class has a lock]

Unlike objects, We can't create multiple classes in the same package.

```java
//Program

package com.ravi.static_synchronization;

class MyTable
{
    public static synchronized void printTable(int num)
    {
        for(int i=1; i<=10; i++)
        {
            IO.println(num+" X "+i+" = "+(num*i));
            try
            {
                Thread.sleep(1000);
            }
            catch(InterruptedException e)
            {
                e.printStackTrace();
            }
        }
        IO.println("=================================");
    }
}

public class StaticSynchronization
{
    public static void main(String[] args)
    {

        Thread t1 = new Thread()
        {
            @Override
            public void run()
            {
                MyTable.printTable(5);

            }

        };

        Thread t2 = new Thread()
        {
            @Override
            public void run()
            {
                MyTable.printTable(10);

            }

        };

        Runnable r1 = ()-> MyTable.printTable(15);
        Thread t3 = new Thread(r1);

        t1.start();  t2.start();   t3.start();

    }

}
```

How to work with synchronized block, If we want to take the lock from the class.

```java
synchronized(MyClass.class) //Taking the lock from the class
{

}
```

Thread Priority :

It is possible in java to assign priority to a Thread. Thread class has provided two predefined methods setPriority(int newPriority) and getPriority() to set and get the priority of the thread respectively.

```java
public final void setPriority(int newPriority) //setter
{
  if (newPriority > Thread.MAX_PRIORITY || newPriority < Thread.MIN_PRIORITY)
     {
        throw new IllegalArgumentException();
     }
    this.priority = newPriority;
}


public final int getPriority() //getter
{
  return this.priority;
}
```

In java we can set the priority of the Thread in numbers from 1- 10 only where 1 is the minimum priority and 10 is the maximum priority.

Whenever we create a thread in java by default its priority would be 5 that is normal priority.

The user-defined thread created as a part of main thread will acquire the same priority of main Thread.

Thread class has also provided 3 final static fields which are as follows :-

```java
  public static final int MIN_PRIORITY = 1;
  public static final int NORM_PRIORITY = 5;
  public static final int MAX_PRIORITY = 10;
```

Note :- We can't set the priority of the Thread beyond the limit(1-10) so if we set the priority beyond the limit (1 to 10) then it will generate an exception java.lang.IllegalArgumentException.

```java
package com.ravi.priority;

public class ThreadPriorityDemo1
{
    public static void main(String[] args)
    {
        Thread t1 = new Thread();
        IO.println("Thread priority is :"+t1.getPriority()); //5

        t1.setPriority(10);
        IO.println("Current thread priority is :"+t1.getPriority());

        t1.setPriority(11); //java.lang.IllegalArgumentException

    }

}
```

```java
package com.ravi.priority;

class Foo extends Thread
{
    @Override
    public void run()
    {
        int priority = Thread.currentThread().getPriority();
        IO.println("Child thread priority is :"+priority); //1
    }
}
public class ThreadPriorityDemo2 {

    public static void main(String[] args)
    {
        Thread t = Thread.currentThread();
        t.setPriority(Thread.MIN_PRIORITY);
        IO.println("Main thread priorigty is :"+t.getPriority()); //1

        Foo f1 = new Foo();
        f1.start();

    }

}
```

Note : If we change main thread priority then child thread will acquire same priority.